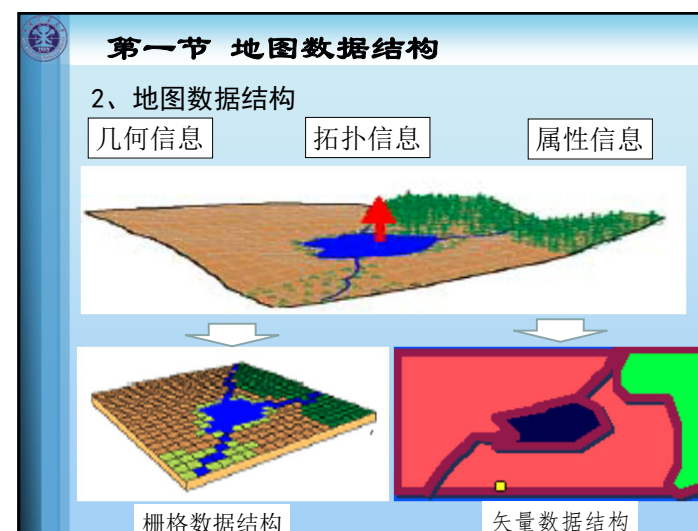
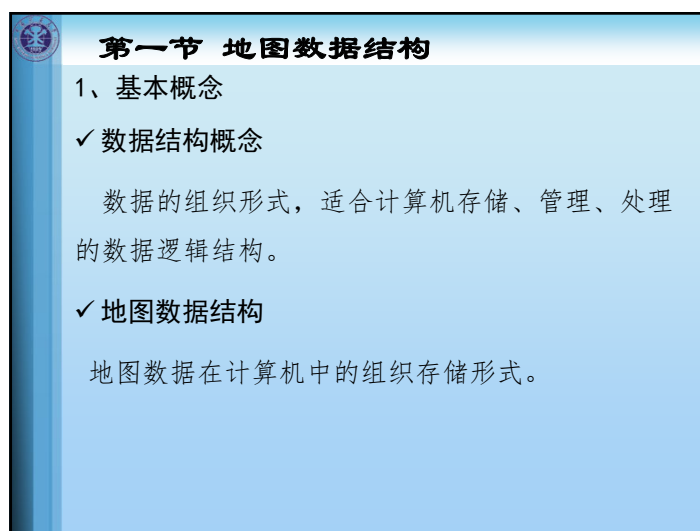
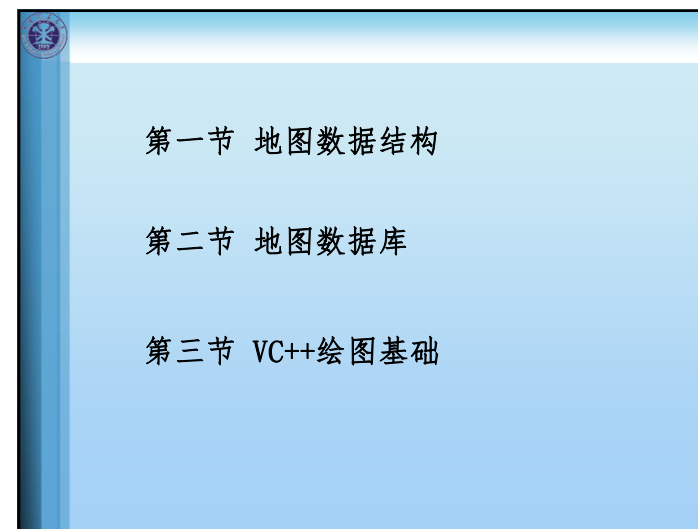
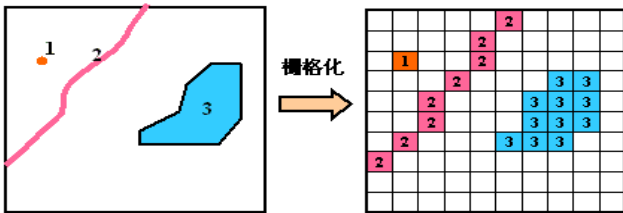




第一节 地图数据结构

3 栅格数据结构

规则的像元阵列来表示地图要素及其分布的数据结构，其阵列中的每个数据表示地物属性特征

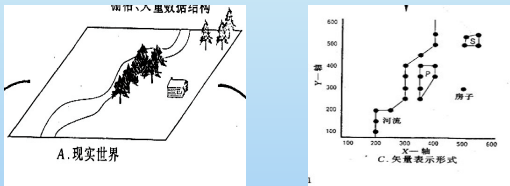


- 1) 点实体——表示为一个像元。
- 2) 线实体——表示为在一定方向上连接成串的相邻像元的集合。
- 3) 面实体——表示为聚集在一起的相邻像元的集合。

第一节 地图数据结构

4 矢量数据结构

通过记录坐标的方式尽可能精确地表达点、线、面分布的地物要素。



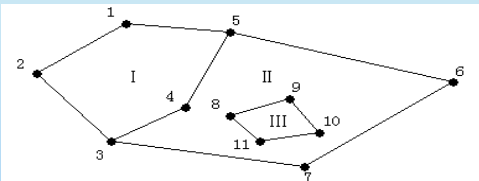
✓ 需记录的信息

- 1) 矢量数据自身的存储和处理；
- 2) 与属性数据的联系；
- 3) 矢量数据之间的空间关系(拓扑关系)；

第一节 地图数据结构

4 矢量数据结构

● 无拓扑关系的矢量结构——Spaghetti模型

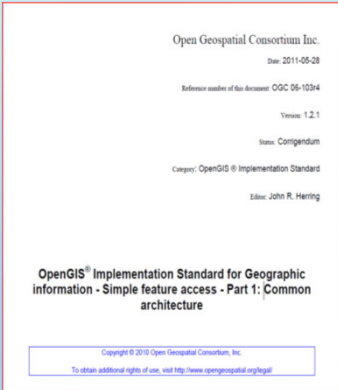


- 1) 点对象：唯一标志码，要素编码， $\langle X, Y \rangle$ 。
- 2) 线对象：唯一标志码，要素编码， $\langle X_1, Y_1 \rangle, \langle X_2, Y_2 \rangle, \dots, \langle X_n, Y_n \rangle$ 。
- 3) 面对象：唯一标志码，要素编码， $\langle X_1, Y_1 \rangle, \langle X_2, Y_2 \rangle, \dots, \langle X_n, Y_n \rangle, \langle X_1, Y_1 \rangle$ 。

多边形	数据项
I	$\langle X_1, Y_1 \rangle, \langle X_2, Y_2 \rangle, \langle X_3, Y_3 \rangle, \langle X_4, Y_4 \rangle, \langle X_5, Y_5 \rangle, \langle X_6, Y_6 \rangle, \langle X_7, Y_7 \rangle, \langle X_8, Y_8 \rangle, \langle X_9, Y_9 \rangle$
II	$\langle X_1, Y_1 \rangle, \langle X_2, Y_2 \rangle, \langle X_3, Y_3 \rangle, \langle X_4, Y_4 \rangle, \langle X_5, Y_5 \rangle, \langle X_6, Y_6 \rangle, \langle X_7, Y_7 \rangle, \langle X_8, Y_8 \rangle, \langle X_9, Y_9 \rangle$
III	$\langle X_1, Y_1 \rangle, \langle X_2, Y_2 \rangle, \langle X_3, Y_3 \rangle, \langle X_4, Y_4 \rangle, \langle X_5, Y_5 \rangle, \langle X_6, Y_6 \rangle, \langle X_7, Y_7 \rangle, \langle X_8, Y_8 \rangle, \langle X_9, Y_9 \rangle$

第一节 地图数据结构

4 矢量数据结构

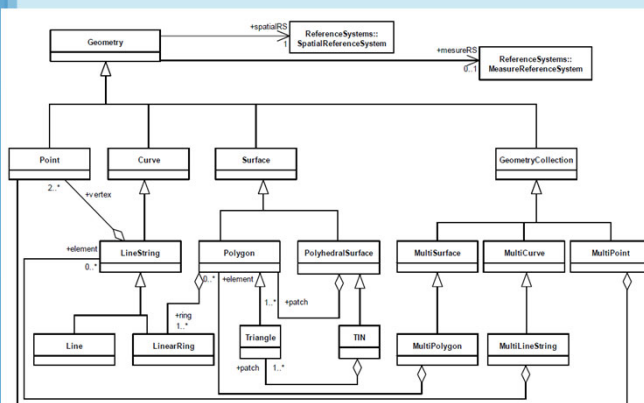


OGC标准

<http://www.opengeospatial.org/standards/sfa>

第一节 地图数据结构

4 矢量数据结构



第二节 地图数据库

1、地图数据库

✓ 地图数据库的地位

地图制图是信息传输过程，也是地图数据处理过程。地图数据处理必须以数据库为中心，以便更有效地实现地图信息采集、存储、检索、分析处理与图形输出等。



地图数据库是数字地图的核心。

第二节 地图数据库

1、地图数据库

✓ 地图数据库的概念

某一区域内一定地图制图要素特征的相关空间数据集合；是地图制图系统在计算机物理存储介质上存储的与应用相关的地图数据的总和，一般是以特定结构的文件的形式组织在存储介质上的。

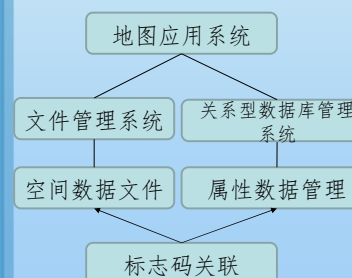
✓ 两种理解方式

- (1) 地图信息的载体：如Coverage及shape文件
- (2) 软件系统：如ESRI ArcSDE+Geodatabase

第二节 地图数据库

2 文件关系数据库混合管理

用一组文件形式来存储地理空间数据及其拓扑关系，利用通用关系数据库存储属性数据，通过唯一的标识符来建立它们之间的连接。



✓ 优点

属性管理能力大大提高

✓ 缺点

空间数据和属性数据分开存储，难以保证数据的存储和操作的统一。

✓ 逐步走出该管理模式

第二节 地图数据库

3 关系数据库管理

图形数据和属性数据统一存储在通用关系数据库中，即将图形数据转成关系存放在关系型数据库二进制块（Binary Block）

多边形号	多边形属性	几何数据
I	...	...
II	...	...

✓ 优点

图属一体管理，避免连接操作

✓ 缺点

变长部分读写效率比定长的属性字段慢得多，涉及对象的嵌套时更慢。

实现SQL查询要附加接口

第二节 地图数据库

3 对象-关系数据库管理

对现有的关系数据库进行扩展，增加空间数据类型支持，使之能直接管理非结构化的空间数据。

空间数据处理

DBMS

空间数据库

扩充实体类型
(点、线、面、圆等)

✓ 优点

解决了空间数据变长记录的存储问题，由数据库软件商开发，效率较高

✓ 缺点

空间数据结构不用用户任意定义，仍受一定限制，没解决对象的嵌套问题

典型代表：Oracle Spatial

第三节 VC++绘图基础

1、VC++工程创建与调试

1.1 工程创建

Visual Studio Installer

已安装 可用

所有安装项都是最新的。

Visual Studio Community 2022

17.9.6

安装已部署

发行说明

更新的

修改(M)

更多

开发人员新闻

Free Pure Virtual C++ 2024 Conference

Pure Virtual C++ is our free one-day virtual conf...

Monday, April 22, 2024

How to use GitHub Copilot Chat in Visual Studio

GitHub Copilot serves as an AI-powered coding s...

Tuesday, April 16, 2024

Introducing the new Copilot experience in Visual Studio

GitHub Copilot your AI-powered coding compen...

Tuesday, April 9, 2024

查看更多 Microsoft 开发新闻。

第三节 VC++绘图基础

1、VC++工程创建与调试

1.1 工程创建

Visual Studio Installer

正在安装 - Visual Studio Community 2022 - 17.9.6

工作负载 单个组件 语言包 安装位置

Web 和云 (4)

ASP.NET 和 Web 开发

用于使用 ASP.NET Core, ASP.NET, HTML/JavaScript 和包括 Docker 支持的部署生成 Web 应用程序。

Python 开发

为 Python 进行编程、调试、交互式开发和源代码管理。

桌面应用和移动应用 (5)

NET Multi-platform App UI 开发

使用 C# 和 .NET MAUI 在单个基于本机运行时生成 Android, iOS, Windows 和 Mac 应用。

使用 C++ 的桌面开发

使用 C++ 和 .NET MAUI 在单个基于本机运行时生成 Android, iOS, Windows 和 Mac 应用。

通用 Windows 平台开发

使用 C# 和 .NET MAUI 在单个基于本机运行时生成 Android, iOS, Windows 和 Mac 应用。

安装详细信息

✓ 适用于最新 v143 生成工具的 C++ MFC 应用

✓ 适用于 v143 生成工具的 C++ 模板化 MFC 应用

✓ Incredibuild - 生成加速

✓ GitHub Copilot

✓ 对 v143 生成工具(模板)的 C++/CLI 支持

✓ 适用于 Windows 的 C++ Clang 工具(17.8...)

✓ IntelliSense 增强

位置

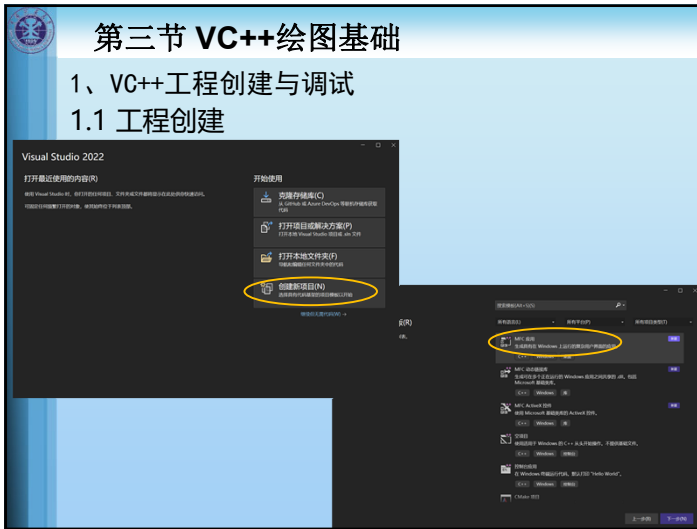
C:\Program Files\Microsoft Visual Studio\2022\Community

要求的总空间 1.23 GB

下载并安装

修改(M)

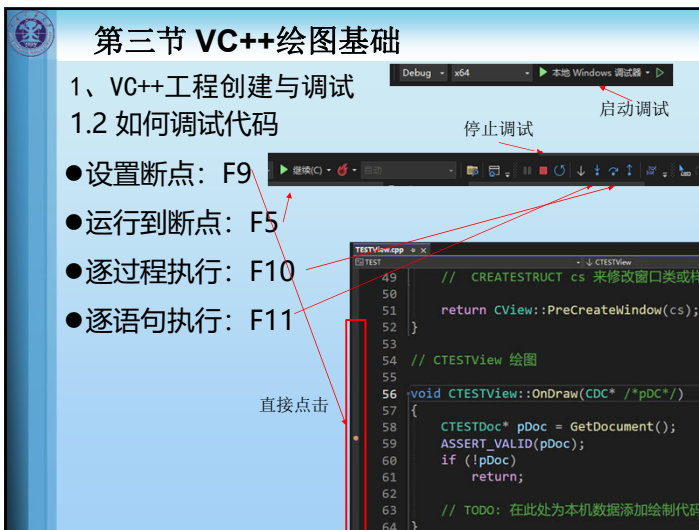
4



第三节 VC++绘图基础

1、VC++工程创建与调试

1.2 如何调试代码



- 设置断点: F9
- 运行到断点: F5
- 逐过程执行: F10
- 逐语句执行: F11

直接点击

```

49 // CREATESTRUCT cs 来修改窗口类或样
50
51 return CView::PreCreateWindow(cs);
52
53 // CTESTView 绘图
54
55 void CTESTView::OnDraw(CDC* /*pDC*/)
56 {
57     CTESTDoc* pDoc = GetDocument();
58     ASSERT_VALID(pDoc);
59     if (!pDoc)
60         return;
61
62     // TODO: 在此处为本机数据添加绘制代码
63 }
64
  
```

第三节 VC++绘图基础

1、VC++工程创建与调试

1.2 如何调试代码



```

55
56 void CTESTView::OnDraw(CDC* /*pDC*/)
57 {
58     CTESTDoc* pDoc = GetDocument();
59     ASSERT_VALID(pDoc);
60     if (!pDoc)
61         return;
62
63     // TODO: 在此处为本机数据添加绘制代码
64 }
  
```

名称: pDoc 值: 0x000001db421b0000

第三节 VC++绘图基础

1、VC++工程创建与调试

1.3 VC++消息机制

✓ 消息 (Message)

关于发生事件的信息。

从程序的观点看，消息驱动就是由消息来驱动/响应某个函数。

✓ Windows是一个基于事件的消息驱动系统

✓ DOS应用程序采用顺序执行过程

第三节 VC++绘图基础

1、VC++工程创建与调试

1.3 VC++消息机制



添加消息处理函数

添加事件处理函数

第三节 VC++绘图基础

2、VC++图形编程基础

2.1 设备环境

设备环境 (Device Context, **DC**)，也称设备上下文，由Windows管理的一个数据结构，保存了绘图操作中一些共同需要设置的信息。

例如，当前画笔、画刷、字体和位图等图形对象及其属性，及颜色和背景等影响图形输出的绘图模式。

形象地说，一个设备环境提供了一张**画布**和一些绘画的**工具**，我们可以使用不同颜色的工具在上面绘制点、线、圆和文本。

第三节 VC++绘图基础

2、VC++图形编程基础

2.2 MFC与CDC类

✓ **MFC**

微软基础类库 (Microsoft Foundation Classes，简称MFC) 是一个微软公司提供的类库，以C++类的形式封装了Windows API。

✓ **CDC类**

MFC中，设备环境 (Device Context, DC) 被抽象为CDC类。

CDC是一个设备环境的基类，MFC还派生了很多子类。

第三节 VC++绘图基础

2、VC++图形编程基础

2.3 OnDraw函数

由系统框架直接调用的，每当窗口重绘时就会自动执行。

```
void Cexp11View::OnDraw(CDC* /*pDC*/)
{
    Cexp11Doc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    if (!pDoc)
        return;
    // TODO: 在此处为本机数据添加绘制代码
}
```

CDC为设备环境类，pDC为CDC类对象的一个指针，可以用它对CDC类的成员函数进行操作。

第三节 VC++绘图基础

2、VC++图形编程基础

2.4 颜色定义

示例: `COLORREF rgb=RGB(192,192,192);`

RGB(r, g, b)是个宏，其中，r、g和b分别表示红、绿、蓝分量值（范围0到255）。

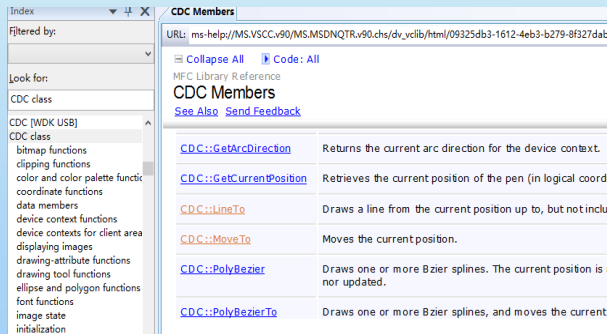
- **COLORREF** 类型是一个32位整数。

颜色	RGB分量值	颜色	RGB分量值
红色	255, 0, 0	深红	128, 0, 0
绿色	0, 255, 0	深绿	0, 128, 0
蓝色	0, 0, 255	深蓝	0, 0, 128
黄色	255, 255, 0	深黄	128, 128, 0
青色	0, 255, 255	深青	0, 128, 128
紫色	255, 0, 255	灰色	192, 192, 192
白色	255, 255, 255	黑色	0, 0, 0

第三节 VC++绘图基础

3、VC++图形类与函数

3.1 如何查找帮助



MSDN

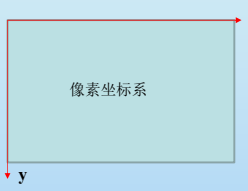
第三节 VC++绘图基础

3、VC++图形类与函数

3.2 点的绘制

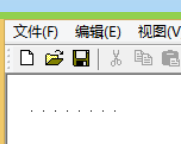
COLORREF CDC::SetPixel(int x, int y, COLORREF crColor);

COLORREF CDC::SetPixel(POINT point, COLORREF crColor);



例如:

```
for(int xPos=20;xPos<100;xPos+=10)
    pDC->SetPixel(xPos,30,RGB(0,0,0));
```



第三节 VC++绘图基础

3、VC++图形类与函数

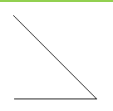
3.3 线的绘制

CPoint CDC:: MoveTo(int x, int y)

BOOL CDC:: LineTo(int x, int y);

例如:

```
pDC->MoveTo(100,100);
pDC->LineTo(200,200);
pDC->LineTo(100,200);
```



第三节 VC++绘图基础

3、VC++图形类与函数

3.4 椭圆及圆的绘制

BOOL CDC::Ellipse(int x1, int y1, int x2, int y2)



例如:

```
pDC->Ellipse(300,100,350,400);
pDC->Ellipse(400,100,500,200);
```



第三节 VC++绘图基础

3、VC++图形类与函数

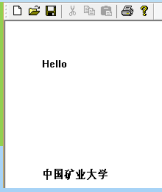
3.5 文本绘制

BOOL CDC::TextOut(int x, int y, const CString& str)

文本输出位置 文本字符串

例如:

```
pDC->TextOut(50, 50, L"Hello");
pDC->TextOut(50, 200, L"中国矿业大学");
```



第三节 VC++绘图基础

3、VC++图形类与函数

3.6 其它图形绘制

```
BOOL Polygon(
    LPPOINT lpPoints,
    int nCount
);
```

多边形区域

```
BOOL Arc(
    LPCRECT lpRect,
    POINT ptStart,
    POINT ptEnd
);
```

圆弧

```
BOOL Rectangle(
    int x1,
    int y1,
    int x2,
    int y2
);
```

矩形

```
BOOL Polyline(
    LPPOINT lpPoints,
    int nCount
);
```

多边形

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.1 概念

图形设备接口 (Graphics Device Interface, **GDI**) 是Windows图形抽象类的接口, 提供基本绘图对象及工具, 如画笔、画刷、字体、位图和调色板等

CGdiObject **MFC中的图形设备接口类**

- CBitmap 位图类
- CBrush 画刷类
- CFont 字体类
- CPalette 调色板类
- CPen 画笔类

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.2 GDI的使用方法

- 1) 创建一个GDI对象


```
CPen PenNew (PS_DASH, 1, RGB(255, 0, 0));
```
- 2) 将GDI对象选入设备环境


```
CPen* pPenOld
pPenOld = pDC->SelectObject(&PenNew);
```

```
CPen* SelectObject(CPen* pPen);
CBrush* SelectObject(CBrush* pBrush);
CFont* SelectObject(CFont* pFont);
CBitmap* SelectObject(CBitmap* pBitmap);
```
- 3) 使用结束后还原原来GDI对象


```
pDC->SelectObject(pPenOld);
```

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.3 创建画笔

```
CPen(
    int nPenStyle,
    int nWidth,
    COLORREF crColor
);
```

样式 线宽 颜色

样 式	说 明
PS_SOLID	实线
PS_DOT	点线
PS_DASH	虚线
PS_DASHDOT	点划线

样 式	说 明
PS_DASHDOTDOT	双点划线
PS_NULL	空的边框
PS_INSIDEFRAME	边框实线

例如:

```
CPen PenNew (PS_DASH, 1, RGB(255, 0, 0));
```

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.3 创建画刷

```
CBrush(
    COLORREF crColor
);
CBrush(
    int nIndex,
    COLORREF crColor
);
```

颜色 模式

- HS_BDIAGONAL Downward hatch (left to right) at 45 degrees
- HS_CROSS Horizontal and vertical crosshatch
- HS_DIAGCROSS Crosshatch at 45 degrees
- HS_FDIAGONAL Upward hatch (left to right) at 45 degrees
- HS_HORIZONTAL Horizontal hatch
- HS_VERTICAL Vertical hatch

例如:

```
CBrush brush1(RGB(255,0,0)); // 创建纯色画刷
CBrush brush2(HS_DIAGCROSS, RGB(0,255,0));
// 创建阴影画刷
```

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.4 创建字体

```
BOOL CreateFontIndirect(
    const LOGFONT* lpLogFont
);
```

```
CFont font;
LOGFONT lf;
memset(&lf, 0, sizeof(LOGFONT)); //将该结构成员初始化为0
lf.lfHeight = 20; //字体高度
_tcsncpy_s(lf.lfFaceName, LF_FACESIZE, L"Arial", 7);
font.CreateFontIndirect(&lf);
CFont* def_font = pDC->SelectObject(&font);
pDC->TextOut(50, 50, L"Hello");
pDC->SelectObject(def_font);
```

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.5 程序示例

第三节 VC++绘图基础

4、图形设备接口 (GDI) 及使用

4.5 程序示例

```
void CXXView::OnDraw(CDC* pDC) {
    CPen PenNew(PS_DASH, 1, RGB(255, 0, 0)); // 创建画笔
    CPen* pPenOld = pDC->SelectObject(&PenNew);
    for(int xPos=20; xPos<100; xPos+=10)
        pDC->SetPixel(xPos, 30, RGB(0, 0, 0)); // 绘制像素点
    pDC->MoveTo(100, 100); // 绘制直线
    pDC->LineTo(200, 200);
    pDC->LineTo(100, 200);
    pDC->Ellipse(300, 100, 350, 400);
    pDC->Ellipse(400, 100, 500, 200);
    pDC->TextOut(50, 50, L"Hello");
    pDC->TextOut(50, 200, L"中国矿业大学");
    pDC->SelectObject(pPenOld);
}
```

实验一 地图数据库与地图绘制

一、实验目的

了解地图数据库基本原理，掌握地图绘制编程方法

二、实验内容

对于给定的地图数据库，将地图数据库中的线要素及点要素读取出来，并绘制到屏幕区域上。

三、实验数据

China.txt			
ID	xLon	yLat	
1	83		
2	ID	xLon	yLat
3	1	10	364
4	2	35	387
5	3	101	429
6	4	97	462
7	5	126	466
8	6	133	498
9	7	165	495
10	8	190	529
11	9	235	487

City.txt					
ID	NAME	Rank	xInt	yInt	
1	北京	1	563	375	
2	天津	2	573	362	
3	石家庄	2	538	344	
4	太原	2	514	341	
5	呼和浩特	2	502	390	
6	沈阳	2	654	407	
7	长春	2	678	442	
8	哈尔滨	2	695	473	
9	上海	2	628	228	
10	南京	2	594	243	
11	杭州	2	612	213	
12	合肥	2	574	240	

实验一 简单地图绘制编程

四、实验提示

- 1) 读取点和线数据
- 2) 利用CDC类的直线和圆绘制函数来绘图

第三节 VC++绘图基础

Qt 6.5.0

Graphics

Graphics in Qt 5 is primarily done either through the imperative QPainter API, or through Qt's declarative UI language, Qt Quick, and its scene graph back-end. Qt 5's graphics capabilities also includes support for printing, as well as the loading and saving of various image formats.

2D Graphics with QPainter

QPainter provides API for drawing vector graphics, text and images onto different surfaces, or QPainterDevice instances, such as QImage, QOpenGLPaintDevice, QWidget, and QPainter. The actual drawing happens in the QPainterDevice's QPainterEngine. The software rasterizer and the OpenGL (ES) 2.0 back-ends are the two most important QPainterEngine implementations. The raster paint engine is Qt's software rasterizer, and is used when drawing on a QImage or QWidget. Its strength over the OpenGL paint engine is its high quality when antialiasing is enabled, and a complete feature set.

- Paint System - Overview over the QPainter classes and architecture.
- Coordinate System - Explains how QPainter's coordinate system works.
- Drawing and Filling - Explains how QPainter performs filling and outlining of vector shapes.

The most important rendering targets for QPainter are:

- QImage - A hardware-independent image representation with direct pixel access. QPainter will use the software rasterizer to draw to QImage instances.
- QPixmap - A image representation suited for display on screen. QPainter will primarily use the software rasterizer to draw to QPixmap instances.
- QOpenGLPaintDevice - A paint device to render to the current OpenGL (ES) 2.0 context. QPainter will use hardware accelerated OpenGL calls to draw to QOpenGLPaintDevice instances.
- QBackingStore - A backbuffer for top-level windows. QPainter will primarily use the software rasterizer to draw to QBackingStore instances.
- QWidget - A baseclass for pre-Qt Quick user interface classes. QPainter will render widgets using a QBackingStore.
- QOpenGLWidget - A painter can also be opened on a QOpenGLWidget. This is provided as a convenience, since technically this is no different than using QOpenGLPaintDevice.

QPainter and related classes are part of the Qt GUI module.

https://wiki.qt.io/Qt_for_Beginners

